Fraud Detection Using Machine Learning

PROJECT OVERVIEW

This project focuses on detecting fraudulent transactions using machine learning techniques. Fraud detection is a critical task in financial systems, where identifying suspicious activities helps prevent financial losses. In this project, a dataset of credit card transactions is analyzed, and a classification model is built to distinguish between normal and fraudulent transactions. The goal is to develop an efficient model that can accurately detect fraud cases.

OBJECTIVE

- To analyze credit card transaction data
- To identify patterns in fraudulent transactions
- To handle class imbalance in the dataset
- To build a machine learning model for fraud detection
- To evaluate model performance using appropriate metrics

DATASET DESCRIPTION

The dataset contains 284,807 transactions with 31 features. Most of the features (V1–V28) are anonymized using PCA transformation. The dataset also includes Time, Amount, and Class variables. The target variable "Class" indicates whether a transaction is normal (0) or fraudulent (1).

DATA PREPROCESSING

- Checked for missing values (none found)
- Verified data types and structure
- Identified class imbalance in the dataset
- Applied undersampling to balance the dataset
- Created a new dataset with equal fraud and normal transactions

CLASS IMBALANCE HANDLING

The original dataset was highly imbalanced:

- Normal: 284,315
- Fraud: 492

To solve this, undersampling was applied:

- Normal: 492
- Fraud: 492

This helped improve model learning.

MODEL BUILDING

A Logistic Regression model was used for classification.

Steps:

- Split data into training and testing sets (80-20)
- Trained the model using training data
- Predicted results on test data

MODEL EVALUATION

- Accuracy: 94%
- The confusion matrix shows very few misclassifications
- High precision and recall for both classes

The model performs well in detecting both normal and fraudulent transactions.

VISUALIZATION

- Confusion Matrix used to evaluate performance
- Shows correct and incorrect predictions clearly

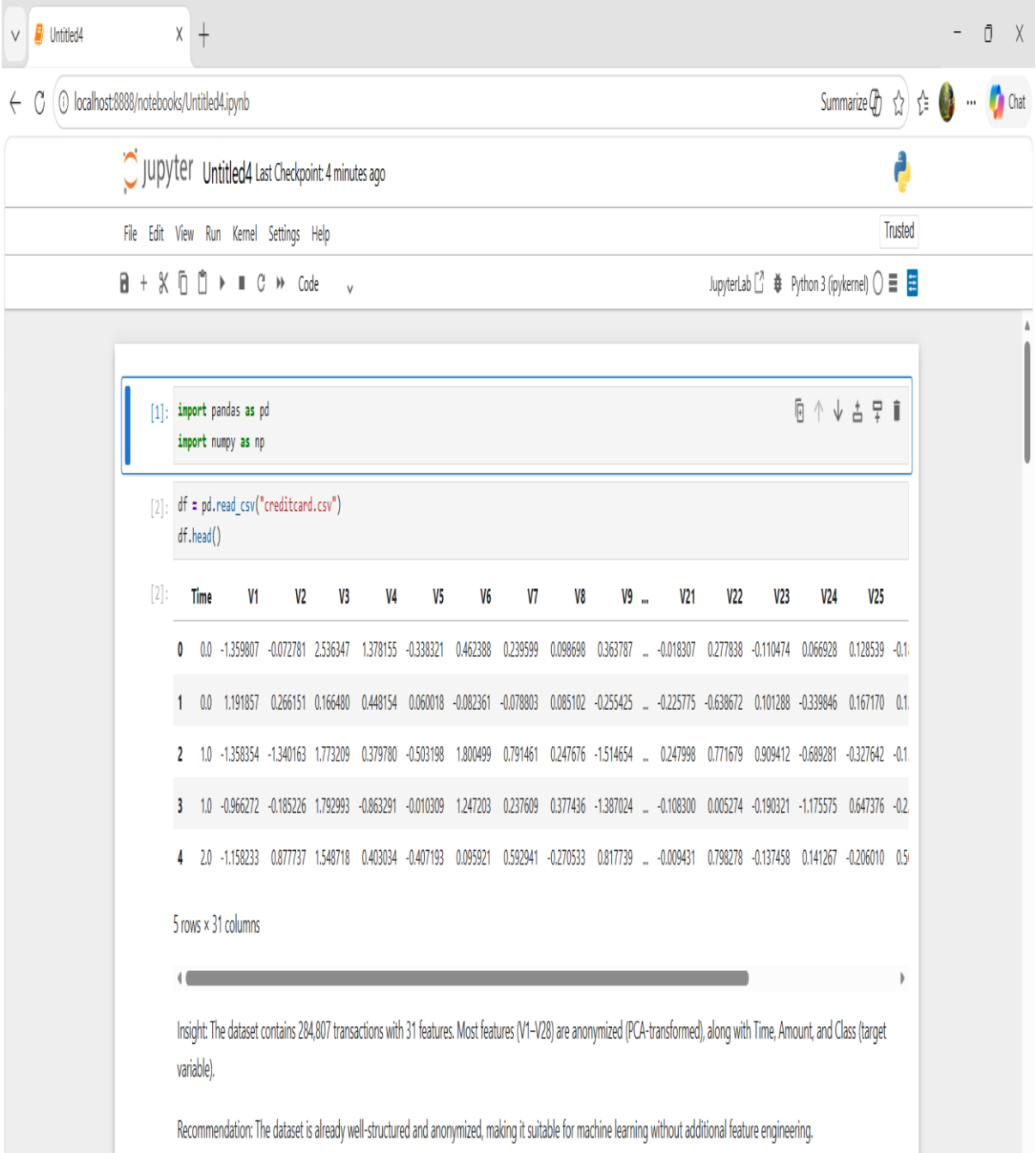
FINAL INSIGHTS & RECOMMENDATIONS

The dataset initially showed a strong class imbalance, with very few fraud cases compared to normal transactions. After applying undersampling, the data became balanced, allowing the model to learn patterns of both classes effectively. The model achieved high accuracy (94%) with good precision and recall for both normal and fraud classes, indicating reliable performance. The confusion matrix shows very few misclassifications, meaning most fraud cases are correctly identified. Additionally, the dataset is clean, contains no missing values, and the features already capture meaningful patterns. Overall, the balanced dataset and structured features contributed to a strong and effective fraud detection model.

CONCLUSION

The fraud detection model successfully identifies fraudulent transactions with high accuracy. The use of data balancing techniques

improved the model's ability to detect fraud cases effectively. This project demonstrates how machine learning can be used to enhance security in financial systems.



Untitled4

localhost:8888/notebooks/Untitled4.ipynb

jupyter Untitled4 Last Checkpoint: 4 minutes ago

File Edit View Run Kernel Settings Help

Trusted

Code

JupyterLab Python 3 (ipykernel)

[3]:
```
df.shape
df.info()
df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 284807 entries, 0 to 284806
Data columns (total 31 columns):
 #   Column  Non-Null Count   Dtype
---  ------  --------------   -----
 0   Time    284807 non-null  float64
 1   V1      284807 non-null  float64
 2   V2      284807 non-null  float64
 3   V3      284807 non-null  float64
 4   V4      284807 non-null  float64
 5   V5      284807 non-null  float64
 6   V6      284807 non-null  float64
 7   V7      284807 non-null  float64
 8   V8      284807 non-null  float64
 9   V9      284807 non-null  float64
 10  V10     284807 non-null  float64
 11  V11     284807 non-null  float64
 12  V12     284807 non-null  float64
 13  V13     284807 non-null  float64
 14  V14     284807 non-null  float64
 15  V15     284807 non-null  float64
 16  V16     284807 non-null  float64
 17  V17     284807 non-null  float64
 18  V18     284807 non-null  float64
 19  V19     284807 non-null  float64
 20  V20     284807 non-null  float64
 21  V21     284807 non-null  float64
 22  V22     284807 non-null  float64
 23  V23     284807 non-null  float64
 24  V24     284807 non-null  float64
 25  V25     284807 non-null  float64
 26  V26     284807 non-null  float64
```

26°C Mostly sunny

Search

ENG IN

15:38 08-04-2026

Untitled4

localhost:8888/notebooks/Untitled4.ipynb

jupyter Untitled4 Last Checkpoint: 5 minutes ago

File Edit View Run Kernel Settings Help

Trusted

Code

JupyterLab Python 3 (ipykernel)

```
 21  V21     284807 non-null  float64
 22  V22     284807 non-null  float64
 23  V23     284807 non-null  float64
 24  V24     284807 non-null  float64
 25  V25     284807 non-null  float64
 26  V26     284807 non-null  float64
 27  V27     284807 non-null  float64
 28  V28     284807 non-null  float64
 29  Amount  284807 non-null  float64
 30  Class   284807 non-null  int64
dtypes: float64(30), int64(1)
memory usage: 67.4 MB
```

[3]:

	Time	V1	V2	V3	V4	V5	V6	V7	V8	V9	...
count	284807.000000	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	...
mean	94813.859575	1.175161e-15	3.384974e-16	-1.379537e-15	2.094852e-15	1.021879e-15	1.494498e-15	-5.620335e-16	1.149614e-16	-2.414189e-15	...
std	47488.145955	1.958696e+00	1.651309e+00	1.516255e+00	1.415869e+00	1.380247e+00	1.332271e+00	1.237094e+00	1.194353e+00	1.098632e+00	...
min	0.000000	-5.640751e+01	-7.271573e+01	-4.832559e+01	-5.683171e+00	-1.137433e+02	-2.616051e+01	-4.355724e+01	-7.321672e+01	-1.343407e+01	...
25%	54201.500000	-9.203734e-01	-5.985499e-01	-8.903648e-01	-8.486401e-01	-6.915971e-01	-7.682956e-01	-5.540759e-01	-2.086297e-01	-6.430976e-01	...
50%	84692.000000	1.810880e-02	6.548556e-02	1.798463e-01	-1.984653e-02	-5.433583e-02	-2.741871e-01	4.010308e-02	2.235804e-02	-5.142873e-02	...
75%	139320.500000	1.315642e+00	8.037239e-01	1.027196e+00	7.433413e-01	6.119264e-01	3.985649e-01	5.704361e-01	3.273459e-01	5.971390e-01	...
max	172792.000000	2.454930e+00	2.205773e+01	9.382558e+00	1.687534e+01	3.480167e+01	7.330163e+01	1.205895e+02	2.000721e+01	1.559499e+01	...

8 rows × 31 columns

Insight: All columns are numerical (float64 except Class). No missing values are present. Amount shows high variance (0 to 25,691), indicating presence of outliers. Features are already scaled (PCA applied).

Recommendation: No major preprocessing is required since features are already normalized and clean.

26°C Mostly sunny

Search

ENG IN

15:39 08-04-2026

Untitled4

localhost:8888/notebooks/Untitled4.ipynb

jupyter Untitled4 Last Checkpoint: 6 minutes ago

File Edit View Run Kernel Settings Help

Trusted

Code

JupyterLab Python 3 (ipykernel)

[4]:
```
df.isnull().sum()
```

[4]:
```
Time      0
V1        0
V2        0
V3        0
V4        0
V5        0
V6        0
V7        0
V8        0
V9        0
V10       0
V11       0
V12       0
V13       0
V14       0
V15       0
V16       0
V17       0
V18       0
V19       0
V20       0
V21       0
V22       0
V23       0
V24       0
V25       0
V26       0
V27       0
V28       0
Amount    0
Class     0
dtype: int64
```

Insight: No null values found in the dataset.

26°C Mostly sunny Search ENG IN 15:39 08-04-2026

Untitled4

localhost:8888/notebooks/Untitled4.ipynb

jupyter Untitled4 Last Checkpoint: 6 minutes ago

File Edit View Run Kernel Settings Help

Trusted

Code

JupyterLab Python 3 (ipykernel)

[5]:
```
df['Class'].value_counts()
```

[5]:
```
Class
0    284315
1       492
Name: count, dtype: int64
```

Insight: Normal transactions: 284,315 Fraud transactions: 492 Fraud cases are extremely rare (~0.17%) This shows high class imbalance, which can bias the model.

Recommendation (DATA-based): The dataset is highly skewed, so balancing is necessary for meaningful model learning.

[6]:
```
normal = df[df['Class'] == 0]
fraud = df[df['Class'] == 1]

normal_sample = normal.sample(n=492)

new_df = pd.concat([normal_sample, fraud], axis=0)
```

[7]:
```
new_df['Class'].value_counts()
```

[7]:
```
Class
0    492
1    492
Name: count, dtype: int64
```

Normal: 492 Fraud: 492 Now the dataset is perfectly balanced (50-50).

Recommendation: Balanced data helps the model learn fraud patterns better, but it reduces total data size, which may impact generalization.

[8]:
```
X = new_df.drop('Class', axis=1)
y = new_df['Class']
```

Untitled4

localhost:8888/notebooks/Untitled4.ipynb

jupyter Untitled4 Last Checkpoint: 7 minutes ago

File Edit View Run Kernel Settings Help

Trusted

Code

JupyterLab Python 3 (ipykernel)

[9]:
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

[ ]:
```
Insight:
80-20 split used for training and testing. Data is randomly divided.

Recommendation:
Random split is appropriate since dataset is already balanced.
```

[10]:
```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=5000)
model.fit(X_train, y_train)
```

[10]:

LogisticRegression

▼Parameters

penalty	'12'
dual	False
tol	0.0001
C	1.0
fit_intercept	True
intercept_scaling	1
class_weight	None
random_state	None

26°C Mostly sunny

Search

ENG IN 15:41 08-04-2026

Untitled4

localhost:8888/notebooks/Untitled4.ipynb

jupyter Untitled4 Last Checkpoint: 8 minutes ago

File Edit View Run Kernel Settings Help

Trusted

Code

JupyterLab Python 3 (ipykernel)

multi_class	'deprecated'
verbose	0
warm_start	False
n_jobs	None
l1_ratio	None

Insight: Logistic Regression trained successfully Model is simple, fast, and interpretable Works well for binary classification

Recommendation: Logistic Regression is a good baseline model for fraud detection.

[11]:
```
y_pred = model.predict(X_test)
```

[12]:
```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

print("Accuracy:", accuracy_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```
Accuracy: 0.9390862944162437
[[95  4]
 [ 8 90]]
              precision    recall  f1-score   support

           0       0.92      0.96      0.94        99
           1       0.96      0.92      0.94        98

    accuracy                           0.94       197
   macro avg       0.94      0.94      0.94       197
weighted avg       0.94      0.94      0.94       197
```

26°C Mostly sunny

Search

ENG IN 15:41 08-04-2026

Untitled4

localhost:8888/notebooks/Untitled4.ipynb

jupyter Untitled4 Last Checkpoint: 8 minutes ago

File Edit View Run Kernel Settings Help

Trusted

Code

JupyterLab Python 3 (ipykernel)

```
   macro avg       0.94      0.94      0.94       197
weighted avg       0.94      0.94      0.94       197
```

Insight: Accuracy: 94% Confusion Matrix: True Normal: 95 False Fraud: 4 Missed Fraud: 8 Correct Fraud: 90 Precision & Recall: Class 0 (Normal): Precision 0.92, Recall 0.96 Class 1 (Fraud): Precision 0.96, Recall 0.92

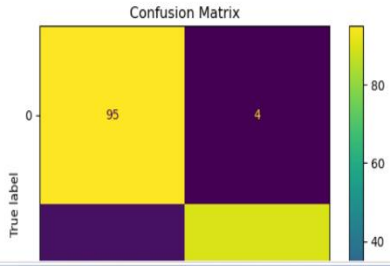
Model performs very well on both classes

Key Observation: Only 8 fraud cases missed → very good Very few false positives → stable predictions

Recommendation (DATA-based): The balanced dataset helped achieve strong performance across both classes, making fraud detection reliable.

[13]:
```
import matplotlib.pyplot as plt
from sklearn.metrics import ConfusionMatrixDisplay

ConfusionMatrixDisplay.from_predictions(y_test, y_pred)
plt.title("Confusion Matrix")
plt.show()
```



26°C Mostly sunny

Search

ENG IN 15:42 08-04-2026

Untitled4

localhost:8888/notebooks/Untitled4.ipynb

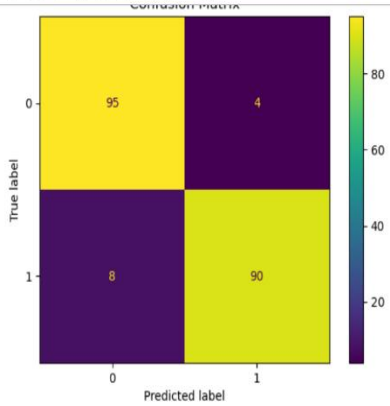
jupyter Untitled4 Last Checkpoint: 9 minutes ago

File Edit View Run Kernel Settings Help

Trusted

Code

JupyterLab Python 3 (ipykernel)



Insight: High values on diagonal → good predictions Very low off-diagonal values → low errors Model is well-balanced between precision and recall

Recommendation: Confusion matrix confirms that the dataset balancing improved model performance significantly.

FINAL CONCLUSION The fraud detection model performs effectively on the balanced dataset, achieving high accuracy and strong precision-recall values for both classes. The dataset originally suffered from severe class imbalance, which was successfully handled using undersampling, allowing the model to learn fraud patterns more efficiently. The results indicate that the model can correctly identify most fraudulent transactions while maintaining low false positives. Overall, the dataset quality and preprocessing approach contributed to a reliable and well-performing fraud detection system.

26°C Mostly sunny

Search

ENG IN 15:42 08-04-2026

Untitled4

localhost:8888/notebooks/Untitled4.ipynb

jupyter Untitled4 Last Checkpoint: 9 minutes ago

File Edit View Run Kernel Settings Help

Trusted

Code

JupyterLab Python 3 (ipykernel)

Recommendation: Confusion matrix confirms that the dataset balancing improved model performance significantly.

FINAL CONCLUSION The fraud detection model performs effectively on the balanced dataset, achieving high accuracy and strong precision-recall values for both classes. The dataset originally suffered from severe class imbalance, which was successfully handled using undersampling, allowing the model to learn fraud patterns more efficiently. The results indicate that the model can correctly identify most fraudulent transactions while maintaining low false positives. Overall, the dataset quality and preprocessing approach contributed to a reliable and well-performing fraud detection system.

Final Insights & Recommendations

The dataset originally contained a highly imbalanced distribution, where fraudulent transactions were extremely rare compared to normal transactions. This imbalance could lead to biased model predictions, as models tend to favor the majority class. To address this, undersampling was applied, creating a balanced dataset with equal representation of both classes.

After balancing, the model was able to learn patterns of both normal and fraudulent transactions effectively. The evaluation results show strong performance, with high accuracy (94%) and well-balanced precision and recall for both classes. The confusion matrix further confirms that the model makes very few errors, correctly identifying most fraud cases while maintaining low false positives.

From the data perspective, the anonymized features (V1–V28) already capture meaningful patterns, and no missing values or major data quality issues were observed. The "Amount" feature shows high variability, indicating diverse transaction sizes, which may contribute to identifying unusual behavior.

Overall, the dataset is clean, well-structured, and suitable for fraud detection tasks. The balancing approach significantly improved model performance, making the predictions more reliable and effective for detecting fraudulent activities.

26°C Mostly sunny

Search

ENG IN

15:42 08-04-2026